

第二章 Unix 系统概貌（下）

同济大学计算机系操作系统课程讲稿

邓蓉

2022-9-10, 2025-10-10修订

3、文件读写操作（Linux为例）

3.1 read系统调用

#include <unistd.h> // Unix的标准库，很多系统调用的钩子函数

```
count = read( fd, &buf, num );
```

读打开文件表中 fd 管理的文件（u_ofiles[fd]），从偏移量 f_offset 开始，连续读 num 个字节；把它们存入起始地址是 buf 的内存区域。read 的返回值 count 是一个整数，有3种情况：

- count == num; 成功读入 num 个字节。
- 0 < count < num; 成功。读进文件尾部 count 个字节。
- count == 0; 读写指针在文件尾部，EOF（End Of File），没有数据可读。
- count == -1; 读操作失败。全局变量 error 是出错原因。

read() 的具体步骤：

- 从读写指针的当前位置（偏移量 f_offset）连续读 n 个字节，写 buf 指向的内存数据块。
- f_offset += n。

3.2 write系统调用

#include <unistd.h> // Unix的标准库，很多系统调用的钩子函数

```
count = write( fd, &buf, num );
```

将内存中的数据写入打开文件表中 fd 管理的磁盘文件（u_ofiles[fd]），从偏移量 f_offset 开始，连续写 num 个字节；buf 是指向数据的指针。write 的返回值 count 是一个整数，有3种情况：

- count == num; 成功写入 num 个字节。
- count == -1; 写操作失败。全局变量 error 是出错原因[备注1]。

write() 的具体步骤：

- buf 指向的内存数据块 → 磁盘文件。从 f_offset 开始，连续写 n 个字节。
- f_offset += n。
- 若写操作导致文件长度增加，修改 FCB 相应字段。

[备注1] error 是库函数中定义的全局变量，是系统调用的出错原因。

3.3 lseek系统调用

#include <unistd.h>

```
off_t lseek(int fd, off_t offset, int whence); // off_t 就是 int
```

显式调整文件的读写指针，设置文件的当前读写位置 f_offset。

- fd: 文件描述符
- offset: 偏移量（字节数，可正可负）
- whence: 基准类型（SEEK_SET/SEEK_CUR/SEEK_END）
 - SEEK_SET: 从文件开头计算
 - SEEK_CUR: 从当前位置计算

- SEEK_END: 从文件末尾计算 (常见于 offset 为负)
- 若成功, 返回调整后的读写指针, 具体值如下:
 - SEEK_SET: offset
 - SEEK_CUR: f_offset + offset
 - SEEK_END: f_size + offset
 如果失败, 返回-1, 并设置errno

3.2 文件的顺序读写操作

不使用 lseek 系统调用, 每次 read/write 从上次读写操作结束的位置开始。下图所示copy程序执行文件顺序读写操作, 将文件file1中的内容复制到file2。

./copy file1 file2

```
#include <fcntl.h>
char buffer[2048];
int version = 1;

int main(argc,argv)
    int argc
    char *argv[];
{
    int fdold,fdnew;
    fdold = open(argv[1], O_RDONLY);
    if(fdold == -1)
    {
        printf("can not open file %s\n",argv[1]);
        exit(1);
    }
    fdnew = creat(argv[2], 0666);
    copyOperation (fdold, fdnew);
    printf ("This is process %d: \n", getpid() );
    printf("Copy Done!From %s To %s\n",
        argv[1],argv[2]);
    exit(0);
}

copyOperation (old,new)
    int old,new;
{
    int count;
    while ((count=read(old,buffer,sizeof(buffer)))>0)
        write(new, buffer, count);
}
```

fdold 是 file1 的文件描述符, 整数 3。open系统调用返回后, file1读写指针f_offset是0。
fdnew 是 file2 的文件描述符, 整数 4。creat系统调用返回后, file2读写指针f_offset也是0。
成功打开这两个文件后, copy程序调用copyOperation函数, 进while循环、执行复制操作。

- 第1轮循环
 - read 系统调用, 读文件 file1, 从当前位置 f_offset 开始, 连续读入2048字节、存进内存 buffer数组, 完成后调整读写指针 f_offset += 2048 = 2048。
 - write系统调用, 将数组buffer中的内容写入file2文件, 从0#字节开始, 写2048个字节这就复制了第1个数据块, file2文件长度增加, 变成2048字节。
- 第2轮循环
 - read 系统调用, 从上次读操作结束的位置开始, 读文件 file1的下一个数据块, 存入buffer数组。

■ write系统调用，将新近读入的数据块写入file2
这就复制了第2个数据块，file2文件长度增加，变成4096字节。

。 。 。

这是文件的顺序读写操作，从文件的某个位置开始，顺序读写后续连续的大数据块。

顺序读写，常用于需要遍历磁盘文件的应用，每块数据用且仅用一次。

3.3 文件的随机读写操作

数据库查询是随机读写。借助索引，DBMS定位目标记录，实施精准读写操作。因为不曾遍历载体文件，所以不是顺序读写。具体实现层面，数据库查询操作主要包括下面3个步骤：

- `off = f( select …… );` // 查索引，定位目标记录在载体文件中的起始偏移量
- `lseek( fd, off, SEEK_SET );` // 显式调整读写指针
- `count = read( fd, &record, sizeof(record) );` // 读入目标记录

总结一下，区分一段代码是顺序读写还是随机读写，主要看是否用到lseek系统调用。前者没有，后者有。顺序读写和随机读写的带宽，顺序读写首字节响应速度是衡量文件系统性能的重要指标，后面会仔细说。

3.5 close (fd)

文件使用完毕要及时关闭。系统会

- 释放指针 `u_ofile[fd]` 引用的系统资源。
- `u_ofile[fd] = null`。

例题：如下两段代码。识别顺序读写操作 和 随机读写操作，写出（1）每次 read 系统调用读入的文件内容（2）所有系统调用完成后，读写指针 `f_offset` 的值。

代码 1

```
fd = open( file1, O_RDONLY ); // f_offset == 0
read( fd, …, X ); // 读入数据块的文件偏移量 ∈ [ 0, X ), f_offset == X
read( fd, …, Y ); // 读入数据块的文件偏移量 ∈ [ X, X+Y ), f_offset == X+Y
read( fd, …, Z ); // 读入数据块的文件偏移量 ∈ [ X+Y, X+Y+Z ), f_offset == X+Y+Z
close( fd );
```

代码 2

```
fd = open( file1, O_RDONLY ); // f_offset == 0
read( fd, …, X ); // 读入数据块的文件偏移量 ∈ [ 0, X ), f_offset == X
lseek( fd, 1000, SEEK_SET ); // f_offset == 1000, 没有读文件
read( fd, …, Z ); // 读入数据块的文件偏移量 ∈ [ 1000, 1000+Z ), f_offset == 1000+Z
close ( fd );
```

总结：（1）打开文件表和相关数据结构是每个进程的文件系统使用现场，记录了正在使用的每个文件，现在都读到哪儿了，还有内存中暂存有文件数据块的缓存块（2）进程使用其打开文件表获取新数据/输出运算结果。read，获取新数据；write，输出运算结果（3）这些数据结构，消耗系统资源，所以不用的文件应该尽快关掉。

习题 参考文献《深入理解计算机系统》

一、

1、fd1, fd2 的值是几？

2、下面程序的输出是什么？解释程序的输出。

```

int main()
{
    int fd1, fd2;

    fd1 = Open("foo.txt", O_RDONLY, 0);
    Close(fd1);
    fd2 = Open("baz.txt", O_RDONLY, 0);
    printf("fd2 = %d\n", fd2);
    exit(0);
}

```

二、下面程序的输出是什么？解释程序的输出。

假设 foobar.txt 文件的内容是字符串“1234567890”

```

1  int main()
2  {
3      int fd1, fd2;
4      char c;

5      fd1 = Open("foobar.txt", O_RDONLY, 0);
6      fd2 = Open("foobar.txt", O_RDONLY, 0);
7      Read(fd1, &c, 1);
8      Read(fd2, &c, 1);
9      printf("c = %c\n", c);
10     exit(0);
11 }

```

四、父进程 和 子进程

Unix系统中，除了0#进程，所有其它进程都是fork出来的。

父进程执行fork系统调用创建子进程。

4.1 fork系统调用

fork()是Linux/Unix系统中用于创建新进程的核心系统调用。函数原型和返回值如下：

```
#include <unistd.h>
```

```
pid_t fork(void);
```

新建的子进程，拥有和父进程完全相同的进程图像：相同的代码段、数据段、用户栈。

- 代码段相同，父子进程执行同一个应用程序，共享正文段。
- 数据段、用户栈相同，子进程继承fork执行时刻父进程所有变量当前值。

为了确保2个进程能够独立运行，为子进程数据段、堆栈段分配内存，复制父进程图像。所以，fork返回后，应用程序的所有变量有2个独立复本，父进程一份、子进程一份。父进程对变量的更新子进程不可见；子进程的更新父进程也不可见。

子进程图像创建成功后，父进程 fork() 返回，返回值是子进程的pid，是一个正整数；新建的子进程被系统选中之后，也从fork()返回，返回值是0。子进程图像创建失败，父进程 fork()返回 -1。

子进程创建成功后，父子进程独立。谁先从fork返回时不一定的。

4.2 fork系统调用的用法

例：这是执行 fork() 系统调用的example程序。请写出程序的执行细节和输出结果。假定父进程pid==3，子进程pid==5。

```
int a=0;
int main( )
{
    int i = -2 ;
    while( ( i=fork( ) ) == -1 );
    if( i )
    {
        a = a+1;
        printf("parent : a = %d ; i = %d\n", &a, &i);
    }
    else
    {
        a=a+4;
        printf("child : a = %d ; i = %d.\n", &a, &i);
    }
    printf( "The end of process %d.\n", getpid( ) );
}
```

父进程执行fork系统调用创建子进程。成功后，系统并发2个example进程。

子进程复制父进程图像，继承父进程变量值，a == 0，i == -2。

子进程创建成功后，fork系统返回。

- 父进程返回子进程的pid号。i 变量是正整数。

- 子进程返回0。i == 0。

if判断，父子进程走不同分支。

- 父进程走红色分支，输出
a = 1; i = 5.
The end of process 3.
- 子进程走金色分支，输出
a = 4; i = 0.
The end of process 5.

关注a变量，父子进程独立处理自己的副本。

- a初值为0。父进程a+1，a变量的值更新为1。子进程的a变量不受影响。
 - 子进程a+4，a变量的值更新为4。父进程的a变量不受影响。
- 父子进程执行的先后次序，不会影响a变量最终的值。

程序的输出是4行，这是因为最后的printf语句，父子进程都会执行。

fork()系统调用成功后，有可能父进程先运行，也有可能子进程先运行。

- 父进程先运行。输出可能有3种情况
 - a = 1; i = 5.
The end of process 3.
a = 4; i = 0.
The end of process 5.
 - a = 1; i = 5.
a = 4; i = 0.
The end of process 3.
The end of process 5.
 - a = 1; i = 5.
a = 4; i = 0.
The end of process 5.
The end of process 3.
- 子进程先运行，输出也可能有3种情况。习题，自己写。

4.3 让子进程分担计算量

- 阅读、编写方阵自乘程序，回答以下问题

```
# include <stdlib.h>
# include <stdio.h>
```

```
# define M 4096
int version = 1;
int matrixOriginal[M][M];
int matrixDes [M][M];
```

```
void produce ( int row, int column )
{
    int i;
    for (i=0;i<M;i++)
        matrixDes[row][column] += matrixOriginal[row][i] * matrixOriginal[i][column];
}
```

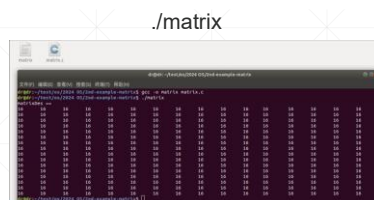
```
int main()
{
    int i, j;
    for (i=0;i<M;i++)
        for (j=0;j<M;j++) {
            matrixOriginal[i][j] = 1;
            matrixDes[i][j] = 0;
        }
}
```

```
for (i=0;i<M;i++)
    for (j=0;j<M;j++)
        produce(i, j);

printf("matrixDes == \n");

for (i=0;i<M;i++) {
    for (j=0;j<M;j++)
        printf("%d\t", matrixDes[i][j]);
    printf("\n");
}
```

1、matrix程序的代码，包括？
2、指出matrix程序的只读数据，全局变量和局部变量。
3、M是什么（是编译器使用的符号，不是应用程序的数据）？请将M修改为4096，测速。为什么程序慢那么多？加速它。



这个程序会跑很久。多核系统中，它只能利用1个CPU核心。
下面尝试 fork 1个子进程，分担父进程运算量。

```

#include <stdlib.h>
#include <stdio.h>

#define M 4096
int version = 1;
int matrixOriginal[M][M];
int matrixDes [M][M];

void produce ( int row, int column )
{
    int i;
    for (i=0;i<M;i++)
        matrixDes[row][column] += matrixOriginal[row][i] * matrixOriginal[i][column];
}

int main()
{
    int i , j;

    for (i=0;i<M;i++)
        for (j=0;j<M;j++) {
            matrixOriginal[i][j] = 1;
            matrixDes[i][j] = 0;
        }

    int child;
    int from, to;
    child = fork()

    if( child > 0 )    // 父进程计算后M/2行
    {
        from = M/2;
        to = M;
        for (i=from;i<to;i++)
            for (j=0;j<M;j++)
                produce(i , j);
        wait( );      // 等待子进程输出结果矩阵的前半部分
    }
    else {             // 子进程计算前M/2行
        from = 0;
        to = M/2;
        for (i=from;i<to;i++)
            for (j=0;j<M;j++)
                produce(i , j);
    }

    for (i=from;i<to;i++) { // 子进程输出结果矩阵的前M/2行; 父进程, 后M/2行
        for (j=0;j<M;j++)
            printf("%d\t", matrixDes[i][j]);
        printf("\n");
    }
}

```

在双核以上的计算机系统中，这个程序可以使用2颗CPU。

大括号括出来的代码实施矩阵运算操作，运行速度是串行版本的2倍。

习题：（1）跑通这个程序（2）用样例程序中的方法，4倍速，8倍速，16倍速……矩阵乘法程序，在你的笔记本上，不使用其它技术，程序还能跑得更快嘛，上限在哪里（3*）结合本学期思考题一，继续加速这个程序。

fork出来的子进程，除了可以分担父进程的计算量，还可以做很多事儿。。。前导知识有限，不再深入。

4.4 让子进程加载、执行另一个应用程序

4.4.1 exec系统调用

进程执行exec系统调用加载目标应用程序。函数原型和返回值如下：

```
#include <unistd.h>
```

```
int execl(const char *path, char *const argv[]);
```

- path是目标应用程序的路径名，可以是绝对路径名，也可以是相对路径名。
- argv是为其提供的命令行参数。第1个参数是程序的文件名，后续字符串是其余命令行参数。

exec系统调用刷新进程图像，主要包含如下3个步骤：

- 抛弃原来的代码段、数据段和用户栈。Process结构和user结构不动。
- 为目标程序的代码段、数据段、用户栈分配内存空间，从磁盘加载目标程序的代码段和数据段。
- 跳转至main函数入口，开始执行目标应用程序。

4.4.2 Unix派发子进程、执行应用程序

在用户界面shell程序中，fork和exec配合使用。需要执行应用程序时，shell就fork一个子进程，让该进程exec，承担执行目标应用程序的任务。

首先，我们编写一个程序tryExec，用来启动另一个程序trivialProg，以此说明exec系统调用的使用方法。

1、tryExec.c，启动trivialProg，为其提供命令行参数。

```

#include <stdio.h>
#include <stdlib.h>
#include <sys.h>

int main1()
{
    int pid;
    char *const argv[] = {"trivialProg\0", "arg1\0", "arg2\0", "and arg3\0"}; // trivialProg 的 4
    个命令行参数

    pid = fork(); // 新建一个子进程
    if (pid == 0)
        // 子进程抛弃原先的进程图像 tryExec。装入新的应用程序。从 main 函数的入口开始执行 trivialProg
        execv ("/trivialProg\0", argv) < 0);
    else
        wait(); // 父进程入睡。等待子进程终止

    printf("The end of tryExec.\n");
}

```

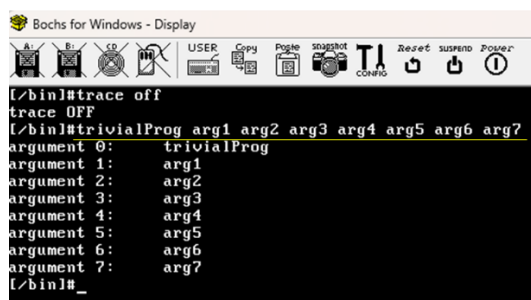
图：tryExec程序加载应用程序trivialProg。argv是提供给trivialProg的命令行参数

tryExec进程执行fork系统调用创建子进程。

fork返回时刻，子进程图像里是tryExec程序的代码和数据。

随后，父进程执行wait系统调用，入睡、等待子进程终止。子进程执行exec系统调用，加载trivialProg程序，屏幕输出tryExec程序提供的命令行参数。

2、trivialProg程序



```

#include <stdio.h>
#include <stdlib.h>

int main1(int argc, char *argv[])
{
    int i;
    for(i = 0; i < argc; i++)
        printf("argument %d:\t%s\n", i, argv[i]);
    exit(0);
}

```

图：trivialProg程序，回显用户输入的命令行

3、动手试。从tryExec到shell。

习题1：修改tryExec程序，派发子进程trivialProg，回显tryExec程序的命令行。具体要求：

- 用命令行tryExec arg1 arg2 arg3启动tryExec程序
- tryExec程序，将这个命令行转化为 trivialProg arg1 arg2 arg3 启动程序 trivialProg
-

习题2：Linux系统，实现shell。简版shell框架如下：

.....

```

While( )
{
    printf("#"); // 输出命令行提示符
    scanf("%s", cmdLine); // 读取用户输入的命令行
}

```

```

parse(cmdLine, argv);      // 解析cmdLine, argv存解析出来的字符串

i = fork(); // 创建一个子进程
if(i)
    wait(); ..... // shell进程睡眠等待子进程终止
else
    exec(argv[0], argv); // 子进程执行命令行指示的应用程序
}

```

五、Unix的进程树

单核Unix/Linux系统有一棵进程树，节点是进程，边是父子关系。

- 树根是0#进程，又名idle进程（idle，系统空转，没有任务可以执行）。负责
(1) 进程调度 (2) idle时系统维护。
- 1#进程是系统监控进程，又名init进程。负责，
(1) 监控终端，启动shell进程（命令行界面程序）
(2) 孤儿进程终止时，回收PCB（孤儿进程，没有父进程的子进程）
- shell进程是人机接口。用户提交作业，shell启动执行作业所需的应用程序，为每个程序
- 其余的普通进程是shell进程的子进程。每个子进程执行一个应用程序。

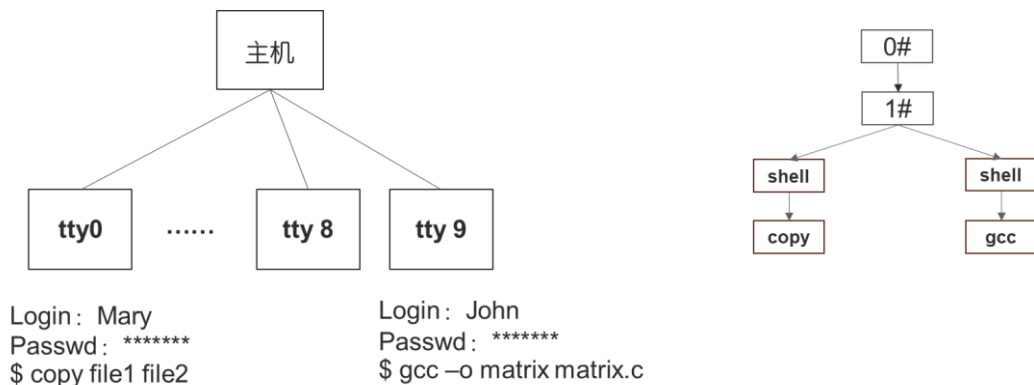


图 1：如左图所示Unix系统运行场景，系统并发6个进程。除0#、1#进程外；2个shell进程分别服务Mary用户和John用户、为他们提供命令行界面；copy进程完成Mary用户提交作业；gcc进程完成John用户提交的作业。右图是进程树。

备注 1：Unix V6++系统，只有一个用户，所以，不设置监控进程。1#进程是shell进程。

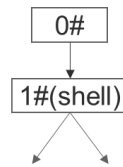


图 2

备注 2：多核Linux，每颗CPU 1个0#进程。除0#进程外，还有若干负责执行系统维护任务的内核线程。如图：

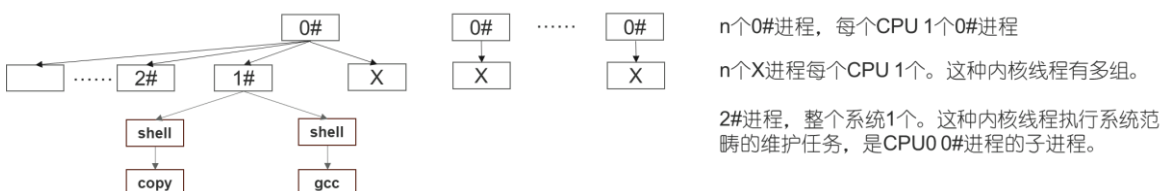


图 3

3、进程的生命周期

进程的生命周期，从fork开始，至应用程序执行完毕结束。

普通进程，应用程序执行完毕终止，生命周期是程序整个执行过程。

shell进程，用户logout结束，生命周期是一次会话；

0#进程 和 1#进程，不会终止。

备注 3：普通进程 和 守护进程 (daemon)

- 普通进程，通常服务于用户提交的作业。应用程序执行完毕，进程终止
- 守护进程，又叫精灵进程，是服务器进程，为上层应用提供服务。
 - 负责维护Linux系统。上图所示所有内核线程，服务Internet请求的inetd
 - 每个应用系统有自己的守护进程，这些进程执行应用程序，服务客户请求。比如，Hadoop的文件系统HDFS，master节点上运行的namenode进程，slave节点上运行的datanode进程。

服务器进程无限循环，永不终止。这些进程初始化系统，随后进入while循环。

4、系统初始化 和 进程树的动态变化过程

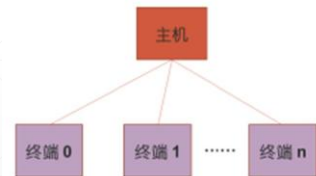
- (1) 将主硬盘上的引导扇区读入内存，执行其中代码，读入操作系统内核映像(kernel.exe)
- (2) 跳转至内核入口main0函数。main0函数初始化系统
 - 读实时钟，得到当前时刻→time变量
 - 初始化硬件设备
 - 初始化所有内核模块
- (3) 创建0#进程
- (4) 0#进程fork创建1#进程
 - 0#进程调用Sched()，入睡，放弃CPU。
 - 1#进程执行应用程序init，开始监控系统。



监控程序 init

- 1#进程为每个终端 fork 1个子进程 Pi。完成后，1#进程睡眠，等待子进程终止。
- 子进程 Pi 管理终端 ttyi
 - (1) 初始化 ttyi：为这个tty 分配 tty 结构（含键盘输入缓存 和 屏幕输出缓存），
 - (2) 设置 p_tty→这个缓存。
 - (3) 设置 标准输入输出文件
 - 0#文件：标准输入文件，引用键盘输入缓存
 - 1#文件：标准输出文件，引用屏幕输出缓存
 - 2#文件：标准错误输出文件，引用屏幕输出缓存
 - (4) exec(login) 执行 login 程序。睡眠、等待用户输入命令行、口令字。

```
login: *****
Password: *****
```





login程序

- 睡眠、等待用户输入命令行、口令字。
- 得到用户输入的用户名、口令字后，查花名册文件/etc/passwd，验证用户身份。

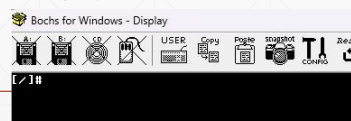
login: ****

Password: ****

user name (in ASCII) (string)	encrypted password	numerical user ID uid	numerical group ID gid	PATH 用户环境变量	initial working directory 工作目录	initial shell (UI program) 用户界面
root	X	0	0	*****	/root	/bin/sh
.....						
John	X	12	1	*****	/home/John	/bin/sh
Mary	X	13	1	*****	/home/Mary	/bin/sh
.....						
Tony	X	58	1	*****	/home/Tony	/bin/x_windows

有匹配记录，验证通过。进程 Pi

- 设置 p_uid = uid
- 设置 u_dirp = 用户的初始工作目录
这是这次会话，相对路径名的搜索起点
- 用户栈底登记环境变量：
环境变量名 = 值，字符串列表
- 执行shell程序为用户提供命令行界面服务，成为shell进程。
- 输出命令行提示符后，shell进程入睡，等待用户输入命令行。



shell程序 (简版框架)

读取用户输入的命令行，启动应用程序

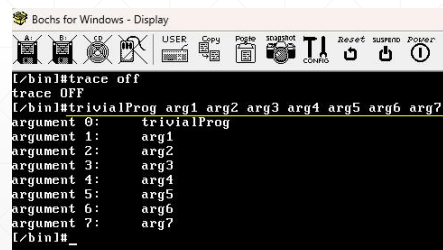
```

While( )
{
    printf("$");
    scanf("%s", cmdLine); // 读取用户输入的命令行

    parse(cmdLine, argv); // 解析cmdLine, argv存解析出来的字符串

    i = fork( ); // 创建一个子进程
    if(i)
        wait( ); ..... // shell 进程睡眠等待子进程终止
    else
        exec(argv[0], argv); // 子进程执行命令行指示的应用程序
}

```



执行应用程序

每执行一个应用程序，就会派发一个子进程

```

While( )
{
    printf("$");
    scanf("%s", cmdLine); // 读取用户输入的命令行

    parse(cmdLine, argv); // 解析cmdLine, argv存解析出来的字符串

    i = fork( ); // 创建一个子进程
    if(i)
        wait( ); ..... // shell 进程睡眠等待子进程终止
    else
        exec(argv[0], argv); // 子进程执行命令行指示的应用程序
}

```

- fork，复制shell进程。
- p_uid, u_dirp, p_ttyp与父进程相同
- 代码段、数据段、用户栈里是shell程序

子进程执行exec系统调用，刷新用户空间。

- p_uid, u_dirp, p_ttyp不变
- 代码段、数据段、用户栈里是目标应用程序

所以，应用程序执行时，操作系统知道执行它的用户，知道它使用的tty终端，知道它的当前工作目录（全是从shell进程那里来的）

- p_uid，用来确定应用程序资源访问权限。
- 当应用程序需要打开文件时，如果使用了相对路径名，u_dirp是目录搜索起点。
- scanf从p_ttyp的输入缓存读数据，printf将字符写入输出缓存供屏幕显示。



tty使用协议

子进程执行期间，shell睡眠，屏幕和键盘归子进程使用

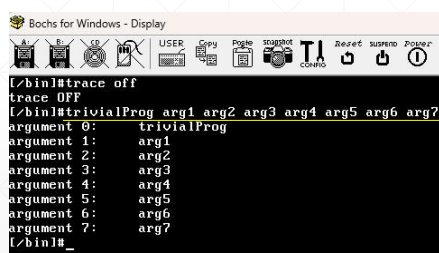
- 子进程读取键盘输入，向屏幕输出。

应用程序终止后，唤醒shell进程。屏幕和键盘归shell进程使用

- shell进程输出命令行提示符，等待用户输入下个命令行。

```
While( )
{
    printf("$");
    scanf("%s", cmdLine); // 读取用户输入的命令行

    // 解析cmdLine, argv存解析出来的字符串
    parse(cmdLine, argv);
    i = fork( );
    if(i)
        wait( ); .....
    else
        exec(argv[0], argv);
}
```



进程终止

- 应用程序执行完毕，子进程终止
 - main返回 或 执行exit系统调用，正常终止
 - 被信号杀死（核心态执行exit函数），异常终止
- shell进程采集终止子进程的运行状态，回收它的PCB
- logout，会话结束，shell进程终止

1#进程采集shell进程的运行状态，回收它的PCB。
为 ttyi 创建1个新进程 (PPT 6)。

```
While( )
{
    printf("$");
    scanf("%s", cmdLine); // 读取用户输入的命令行
    // 解析cmdLine, argv存解析出来的字符串
    parse(cmdLine, argv);

    Switch(argv[0])
    case "cd": shell进程改变当前工作目录
    case "logout": shell进程终止
    shell程序的其它内部命令

    i = fork( );
    if(i)
        wait( ); .....
    else
        exec(argv[0], argv); // 应用程序, shell的外部命令
}
```